TÌM KIẾM CỤC BỘ (LOCAL SEARCH)

1. Mục tiêu

- Hiểu đặc trưng tìm kiếm cục bộ, hàm mục tiêu, trạng thái lân cận và hiện tượng cực trị cục bộ (local optimum).
- Cài đặt, khảo sát Simple Hill Climbing, Steepest-ascent Hill Climbing, Stochastic/First-choice Hill Climbing và Random Restart.
- Ứng dụng Simulated Annealing và khảo sát ảnh hưởng của lịch giảm nhiệt độ đối với chất lượng lời giải.
- Mô hình hóa và giải các bài toán mê cung, n quân hậu (n-Queens) và người du lịch (TSP) bằng tìm kiếm cục bộ.
- Đánh giá kết quả theo hàm mục tiêu, tỉ lệ thành công, số bước lặp, thời gian chạy và khả năng mở rộng; khảo sát tối ưu tính toán với Numba.
- Kiến thức cần nhớ

Thuật toán	Quy tắc lựa chọn	Đặc điểm
Simple Hill Climbing	Chọn bước cải thiện theo thứ tự xét	Đơn giản; dễ mắc kẹt tại cực trị cục bộ.
Steepest-ascent	Chọn lân cận cải thiện tốt nhất	Chất lượng bước tốt; nhiều lượt đánh giá.
Stochastic Hill Climbing	Chọn ngẫu nhiên trong các bước cải thiện	Tăng đa dạng đường đi; có thể vẫn mắc kẹt.
First-choice Hill Climbing	Sinh ngẫu nhiên lân cận cho đến khi gặp bước tốt	Tiết kiệm khi nhiều lân cận.
Random Restart	Chạy lại từ nhiều trạng thái ban đầu	Tăng cơ hội tìm nghiệm tốt/toàn cục.
Simulated Annealing	Có thể chấp nhận bước xấu với xác suất phụ thuộc T	Thoát cực trị cục bộ; nhạy với lịch nhiệt.

2. Dữ liệu và chương trình sử dụng

Nhóm	Tập tin	Nội dung
Minh họa mê cung	Maze_Example_LocalSearch.ipynb	Simple Hill Climbing, khoảng cách Manhattan, đường đi và bẫy cực trị cục bộ.
Dữ liệu/hỗ trợ	L_maze.txt; maze_helper.py	Bản đồ mê cung, hàm phân tích/hiển thị và animation.
Bài tập chính	n_queens.ipynb	Steepest-ascent, Stochastic, Random Restart, Simulated Annealing; đánh giá hội tụ.
Tối ưu hóa	n_queens_fast_conflict_calculation_with_numba.ipynb	So sánh đếm xung đột $O(n^2)$, $O(n)$, JIT Numba.
Lịch nhiệt độ	Simulated_Annealing_Schedules.ipynb	Lịch tuyến tính, mũ, fast và logarithmic.
Bài tập mở rộng	traveling_salesman_problem.ipynb	TSP, hàm tour_length(), các biến thể local search và SA.
Hướng dẫn	README.md	Phân loại notebooks Examples và Exercises.

3. Bài tập thực hành

Bài 1. Tìm kiếm cục bộ trên mê cung - Simple Hill Climbing

- Mở Maze_Example_LocalSearch.ipynb; chạy các cell thiết lập, đọc L_maze.txt và hiển thị bản đồ bằng maze_helper.
- Quan sát các hướng N, E, S, W và hàm expand(maze, pos) sinh trạng thái lân cận hợp lệ.
- Phân tích hàm manhattan(pos1, pos2) và local_search(maze, vis, animation, verbose). Chạy chế độ animation để quan sát quá trình.

© DNTAI

- Ghi nhận kết quả từ result["solution"], vị trí dừng và so sánh với đích G. Giải thích vì sao thuật toán có thể dừng dù chưa đến đích.
- Thử thay thứ tự xét hướng hoặc lựa chọn ngẫu nhiên (theo cấu hình ORDER/RANDOMIZATION trong notebook), chạy nhiều lần và nhận xét.

```
h(n) = |r_n - r_G| + |c_n - c_G|   (giảm h(n) là cải thiện)
```

Bài 2. Mô hình hóa bài toán n quân hậu

- Mở n_queens.ipynb. Khảo sát các hàm random_board(n), show_board(board) và conflicts(board).
- Biểu diễn trạng thái bằng vector có n phần tử; mỗi cột đặt đúng một quân hậu, giá trị của phần tử là chỉ số hàng.
- Xác định phép di chuyển cục bộ: chuyển một quân hậu sang hàng khác trong cùng cột; đếm số trạng thái láng giềng.
- Thực hiện trên n = 4 và n = 8; so sánh bàn cờ ngẫu nhiên với cấu hình [1, 3, 0, 2] (n = 4).

```
f(board) = conflicts(board) = số cặp quân hậu tấn công nhau
Mục tiêu: min f(board); lời giải hợp lệ khi f(board) = 0.
```

Bài 3. Steepest-ascent và Stochastic Hill Climbing (n-Queens)

- Hoàn thành Task 1 (Steepest-ascent): đánh giá tất cả nước đi lân cận, chọn trạng thái cải thiện tốt nhất; dừng khi không còn cải thiện.
- Hoàn thành Task 2 (Inefficient Stochastic): liệt kê tất cả nước đi cải thiện rồi chọn ngẫu nhiên một nước đi.
- Hoàn thành Task 3 (Efficient Stochastic): tạo từng lân cận ngẫu nhiên và chấp nhận nếu tốt hơn; giới hạn số lần thử để tránh lặp vô hạn.
- Đo số bước lặp, số lần đánh giá conflicts(), giá trị f cuối cùng và thời gian chạy; giải thích đánh đổi giữa chất lượng và chi phí tính toán.
- Chú ý: notebook dành các ô “Code and description go here” để sinh viên tự lập trình, không phải mã lời giải có sẵn.

Bài 4. Random Restart và Simulated Annealing (n-Queens)

- Hoàn thành Task 4: khởi tạo lại bàn cờ ngẫu nhiên tối đa 100 lần cho ba biến thể Hill Climbing; lưu giá trị f tốt nhất và số lần khởi động.
- Hoàn thành Task 5: cài đặt Simulated Annealing với xác suất chấp nhận bước xấu hơn phụ thuộc nhiệt độ T.
- Đặt seed, ngân sách vòng lặp và điều kiện dừng nhất quán; khảo sát tác động của nhiệt độ đầu và tốc độ làm nguội.
- Chạy lặp lại nhiều lần trên n = 4, 8 để ước tính tỉ lệ tìm được lời giải không xung đột.

```
Với Δ = f(mới) − f(hiện tại):
Nếu Δ < 0: chấp nhận; nếu Δ > 0: chấp nhận với p = exp(−Δ/T).
```

Bài 5. Khảo sát lịch giảm nhiệt độ

- Mở Simulated_Annealing_Schedules.ipynb; chạy các hàm temperature_exponential, temperature_linear, temperature_fast, temperature_logarithmic.
- Vẽ đồ thị T(t), sử dụng cùng nhiệt độ ban đầu và khoảng thời gian để so sánh; chú ý logarithmic được định nghĩa với t > 0.
- Thay đổi alpha, eta và T0 rồi nhận xét lịch giảm nhiệt nào làm thuật toán chấp nhận các bước xấu lâu hơn.
- Thử ít nhất hai lịch nhiệt trong Simulated Annealing ở Bài 4 và đối chiếu chất lượng nghiệm, thời gian.

```
Mũ: T(t)=T0·α^t; tuyến tính: T(t)=T0−ηt
Fast: T(t)=T0/(1+t); logarithmic: T(t)=T0/ln(1+t), t>0
```

Bài 6. Tối ưu hàm đếm xung đột bằng Numba

© DNTAI

- Mở n_queens_fast_conflict_calculation_with_numba.ipynb; đối chiếu conflicts_naive(), conflicts() và conflicts_jit().
- Giải thích cách dùng số lượng quân hậu theo hàng/đường chéo và công thức C(k,2) để tránh so sánh từng cặp quân hậu.
- Chạy phép đo thời gian trên nhiều kích thước bàn cờ; gọi conflicts_jit() một lần trước khi đo để tránh gộp thời gian biên dịch JIT.
- Vẽ đồ thị thời gian chạy và so sánh $O(n^2)$ với O(n), đồng thời nêu khác biệt giữa độ phức tạp thuật toán và lợi ích biên dịch JIT.

```
C(k,2) = k(k-1)/2; f(board) = tổng C(số quân cùng hàng/đường chéo, 2)
```

Bài 7. Tìm kiếm cục bộ cho Traveling Salesman Problem (TSP)

- Mở traveling_salesman_problem.ipynb; nghiên cứu random_tsp(n), random_tour(n), tour_length(tsp, tour) và show_tsp(tsp, tour).
- Xác định trạng thái là hoán vị các thành phố; hàm mục tiêu là tổng độ dài chu trình khép kín.
- Hoàn thành các phần: Steepest-ascent Hill Climbing, Random Restarts, Stochastic Hill Climbing, First-choice Hill Climbing và Simulated Annealing.
- Tự chọn, mô tả rõ toán tử lân cận hợp lệ cho TSP (ví dụ hoán đổi hai thành phố hoặc đảo đoạn 2-opt); giữ cùng toán tử khi so sánh thuật toán.
- Khảo sát thời gian chạy, số thành phố và chiều dài tour tốt nhất; vẽ tour trước/sau tối ưu. Phần Genetic Algorithm là bài mở rộng (bonus).

Bài 8. Tổng hợp thí nghiệm và phân tích

- Hoàn thành Task 6 trong n_queens.ipynb: so sánh thuật toán trên n=4 và n=8 bằng nhiều lần chạy; ghi thời gian và giá trị hàm mục tiêu.
- Vẽ đường hội tụ f(board) theo số vòng lặp của mỗi thuật toán; nhận xét cải thiện ban đầu và hiện tượng dừng sớm.
- Khảo sát khả năng mở rộng với n=4, 8, 12, 16, 20 cho ít nhất hai thuật toán; lập đồ thị log-log thời gian theo n.
- Phần nâng cao: thử toán tử di chuyển khác cho n-Queens; trình bày tác động lên kích thước lân cận và hiệu quả tìm kiếm.

4. Hướng dẫn thực hiện

4.1. Quy trình chạy notebook

1. Giải nén trọn vẹn thư mục lab03 - local search; giữ L_maze.txt và maze_helper.py cùng thư mục notebook mê cung.

2. Mở Jupyter Notebook/JupyterLab hoặc VS Code với Python kernel. Cài đặt các thư viện sử dụng trong từng notebook: numpy, matplotlib, pandas, scipy, numba, ipython (nếu còn thiếu).

3. Chạy cell từ trên xuống, kiểm tra đầu ra trực quan; với notebook n-Queens và TSP, điền mã nguồn vào các cell Task còn trống.

4. Trên Google Colab, tải các file phụ trợ lên cùng thư mục làm việc; kiểm tra các cell mount Google Drive/đổi thư mục trước khi chạy.

5. Cố định random seed khi cần tái lập; thử nhiều seed để đánh giá tính ngẫu nhiên; đo thời gian với cùng điều kiện.

6. Ghi nhận kết quả và phân tích bằng Markdown ngay trong notebook, tránh chỉ nộp mã nguồn không có nhận xét.

4.2. Bảng ghi kết quả thí nghiệm

Bài toán	Thuật toán / cấu hình	Kích thước	f tốt nhất	Tỉ lệ thành công	Vòng lặp	Thời gian (ms)

© DNTAI

n-Queens	Steepest-ascent	4 / 8				
n-Queens	Stochastic (all moves)	4 / 8				
n-Queens	Stochastic (random neighbor)	4 / 8				
n-Queens	Random Restart	8				
n-Queens	Simulated Annealing	8				
TSP	Steepest-ascent	Tự chọn		—		
TSP	Random Restart / SA	Tự chọn		—		

Gợi ý: với n-Queens, tỉ lệ thành công = số lần đạt conflicts = 0 / tổng số lần chạy; với TSP, so sánh tour_length và mức cải thiện tương đối so với tour ban đầu.

4.3. Câu hỏi thảo luận

1. Tìm kiếm cục bộ khác tìm kiếm theo đường đi (BFS, A*) ở mục tiêu và dữ liệu cần lưu trữ như thế nào?
2. Vì sao Simple Hill Climbing có thể mắc kẹt ở local optimum, plateau hoặc ridge?
3. So sánh Steepest-ascent, Stochastic (liệt kê mọi bước tốt) và First-choice theo số lần đánh giá hàm mục tiêu.
4. Random Restart cải thiện xác suất tìm lời giải như thế nào? Khi nào tăng số lần restart không còn hiệu quả?
5. Vì sao Simulated Annealing có thể chấp nhận nghiệm xấu hơn? Tác động của nhiệt độ đến khám phá và khai thác?
6. Lịch tuyến tính, hàm mũ, fast và logarithmic khác nhau ra sao? Lịch nào phù hợp trong thí nghiệm?
7. Vì sao biểu diễn n-Queens mỗi cột đúng một quân giúp giảm không gian tìm kiếm?
8. Vì sao đếm xung đột theo hàng/đường chéo có thể có độ phức tạp O(n) thay vì $O(n^2)$?
9. Khi dùng Numba, vì sao phải loại trừ thời gian biên dịch lần đầu trong phép đo tốc độ?
10. Với TSP, thay toán tử swap bằng 2-opt có thể ảnh hưởng đến lời giải và số láng giềng như thế nào?

5. Yêu cầu sản phẩm nộp

- Notebook n_queens.ipynb và traveling_salesman_problem.ipynb đã hoàn thành các Task chính; các notebook minh họa có chú thích/nhận xét kết quả.
- Bảng tổng hợp thí nghiệm n-Queens (ít nhất n=4, 8) và TSP; kết quả lặp nhiều lần khi thuật toán có yếu tố ngẫu nhiên.
- Ít nhất 3 hình minh họa: mê cung Hill Climbing; đồ thị hội tụ n-Queens; biểu đồ lịch nhiệt hoặc đường tour TSP.
- Phần nhận xét 1–2 trang về local optimum, Random Restart, SA, thời gian, khả năng mở rộng và hạn chế của các thuật toán.
- Notebook chạy được từ đầu, có seed/thiết lập thí nghiệm, không chứa lỗi hoặc đường dẫn tuyệt đối phụ thuộc máy.
- Báo cáo có trang bìa, mục lục; ghi họ tên, mã sinh viên và mức đóng góp của từng thành viên nhóm.

---o0o---

(Hết)

© DNTAI